

DMEI: Dynamic Masking Embedded Indexing Ephemeral-Entropy Indexing for OS Dispatch Tables

Eyal Braun

Abstract

We study integrity protection for command dispatch tables used in operating system kernels, interpreters, and embedded runtimes. These structures map commands or identifiers to function handlers and are critical to correct execution — corruption redirects control flow and compromises system integrity.

We introduce **ephemeral-entropy indexing**: a construction principle in which storage positions depend on a value destroyed during construction. DMEI (Dynamic Masking Embedded Indexing) instantiates this principle as a cuckoo hash table providing: (1) inline tamper detection at probability $1 - 2^{-32}$, no MAC; (2) index obfuscation — a write-only adversary cannot locate a specific entry without a read primitive; (3) $O(1)$ worst-case lookup; (4) session binding at probability $1 - 2^{-32}$.

DMEI targets OS command dispatch tables — kernel syscall tables, interpreter opcode tables, and embedded handler routers — where Rowhammer and limited write primitives are documented attack classes. DMEI achieves all four properties at **12.4 ns median** on an Intel i7-14700KF: $1.75\times$ a plain cuckoo table, $63\times$ faster than MAC-authenticated alternatives, and **21.5% of the total measured kernel syscall dispatch path**.

Index Terms

ephemeral-entropy indexing, OS command dispatch, Rowhammer, tamper detection, cuckoo hashing, index obfuscation, write-only adversary, kernel security, control-flow integrity

I. INTRODUCTION

A. A New Principle: Ephemeral-Entropy Indexing

Consider a data structure whose entries are stored at positions that depend on a value A that exists only during BUILD; after BUILD completes, A 's stack slot is overwritten with an independent ASLR-random decoy address, and the original value of A is no longer recoverable. An adversary wanting to corrupt entry X must know where X is stored. Without a memory-read primitive to recover A from the structure, the adversary faces a uniform distribution over all possible positions — targeted corruption requires first escalating to a read+write attack.

We call this **ephemeral-entropy indexing**: using a transient, irretrievable value as the randomness source for storage placement. DMEI is its first instantiation as a hash table, motivated by a concrete and documented security problem in OS kernels.

Destruction guarantee. After BUILD computes $ARM(A)$ from ephemeral stack address A , A is overwritten with an independent ASLR-random decoy address before scope exit, followed by a mandatory compiler memory barrier. This overwrite is verified to survive -O2 compilation: the compiled assembly contains two distinct writes to A 's stack slot — the original and the decoy (confirmed via gcc -S on the test system). An attacker reading the dead stack frame recovers the decoy — statistically independent of the original A — and cannot reconstruct $ARM(A)$.

1.2 The Problem: Command Dispatch Tables Under Write-Only Attack

Command dispatch tables are ubiquitous in system software. Three concrete instantiations motivate this work:

- **Kernel syscall tables.** The Linux x86-64 syscall table maps 448 syscall numbers to `sys_*` handler function pointers. It remains writable after boot to support loadable kernel modules.
- **Interpreter opcode tables.** Language runtimes (CPython, LuaJIT, SpiderMonkey) maintain opcode-to-handler dispatch tables built at interpreter startup and accessed on every bytecode instruction.
- **Embedded command routers.** Real-time operating systems (FreeRTOS, Zephyr) and DO-178C-certified flight controllers route command identifiers to interrupt service handlers on the critical path of every task switch.

The attacker's goal: corrupt a single dispatch table entry to redirect control flow to attacker-controlled code, without triggering detection, and without a memory-read primitive. An attacker with a write-only primitive — Rowhammer,

DMA write, or limited heap overflow — can compute the target address in a standard hash table and flip bits there. DMEI removes this capability.

1.3 Why Existing Defenses Are Insufficient

- **__ro_after_init** — inapplicable to tables modified after boot.
- **LKRG** — periodic checks only; DMEI detects at lookup time with zero interval.
- **kCFI/FineIBT** — catches exploitation of a corrupted pointer, not the corruption itself. A pointer redirected to a valid CFI target that is merely less-privileged bypasses CFI entirely.
- **Keyed hashing alone** — a keyed hash can hide placement only while the key remains secret and stable; with no inline integrity token, blind writes still become silent corruption opportunities when a target slot is hit. DMEI couples keyed mixing with per-entry consistency checks and bounded lookup semantics.
- **ASLR-only schemes** — ASLR randomizes addresses but does not bind entry correctness at lookup time. Once an attacker can infer or hit an address, ASLR alone provides no inline tamper signal. DMEI adds lookup-time integrity checks and session binding.
- **ECC RAM** — absent from most consumer hardware; multi-bit Rowhammer bypasses single-bit ECC.
- **MAC per entry (e.g., HMAC per slot)** — 100% tamper detection, but 787 ns per lookup on the test system — $63\times$ slower than DMEI and architecturally incompatible with the kernel dispatch path.

1.4 Contributions

- **Ephemeral-entropy indexing** — a construction that enforces write-only attack resistance in dispatch tables.
- **DMEI** — a fully correct cuckoo hash table providing all four security properties at $O(1)$ cost.
- **Formal security theorems** — Theorems 1–7 with complete proofs.
- **Honest analysis** — explicit limits on scope: pointer table slot determinism, read+write adversary boundary, ASLR dependency, Spectre caveat.
- **Verified implementation** — correct cuckoo insertion, compiled assembly verification of decoy overwrite, 100% tamper detection confirmed.
- **Measured syscall context** — DMEI constitutes 21.5% of the real kernel syscall dispatch path on measured hardware.

2. Background

2.1 Dispatch Table Sizes in Real Systems

System	Table Type	Entry Count	Reference
Linux x86-64	syscall table	448 entries	arch/x86/.../syscall_64.tbl
CPython 3.12	opcode dispatch	~119 opcodes	Python/ceval.c computed_gotos
LuaJIT 2.1	bytecode dispatch	97 opcodes	lj_vm.dasc BC_* definitions
seL4 microkernel	capability invocations	~50–200 caps	seL4 Reference Manual §3
FreeRTOS	ISR command table	16–64 handlers	Application-defined
DO-178C flight ctrl.	command router	32–128 handlers	Avionics domain practice
DMEI design range	all of above	$n \leq 500$	covers every row above

Table 1 — Real dispatch table sizes. Every target system falls within DMEI's $n \leq 500$ design range.

2.2 Rowhammer and the Write-Without-Read Property

Kim et al. [2014] showed DDR3 DRAM suffers disturbance errors: hammering two aggressor rows induces bit flips in adjacent victim rows that are never directly accessed. This is not a modeling assumption — it is a physical property of DRAM cell coupling. The Rowhammer attack is write-without-read by construction. Seaborn and Dullien [2015] achieved full kernel privilege escalation (CVE-2015-0565) by locating target PTEs via timing — feasible because the target was at a deterministically computable address. DRAMMER [van der Veen 2016] replicated this on ARM. TRRespass [Frigo 2020] demonstrated Rowhammer on DDR4. If the storage address is not computable without a read primitive, the attacker is reduced to blind hammering: random, unreliable, and far more detectable.

2.3 Syscall Dispatch Performance Budget

The Linux x86-64 syscall entry path takes ~100–200 ns end-to-end on bare metal. On the test system (Intel i7-14700KF, WSL2/Hyper-V), we measured round-trip syscall latency at $p50 = 57.6$ ns for `SYS_getpid` — the closest proxy for pure dispatch cost. DMEI at 12.4 ns constitutes 21.5% of this measured path. HMAC at 787 ns would consume $13.7\times$ the entire measured dispatch path, making it architecturally incompatible. All microbenchmarks used `rdtsc+lfence` serialization with 26-cycle overhead subtracted; end-to-end bare-metal kernel module measurement is future work.

3. Threat Model

The central concern about this work's threat model is: why would an attacker lack a read primitive? This section answers that question precisely, by grounding each adversary model in documented attack classes where the read/write asymmetry is a physical or architectural property — not an assumption.

3.1 Model A — Fault Adversary (Read-Free by Physics)

- **Rowhammer.** The victim row is flipped without being accessed — the attacker never issues a load to the victim address. Write-without-read by construction.
- **Cosmic rays / alpha particles.** Single-event upsets flip bits without any attacker-issued memory access. Relevant for DO-178C safety-critical contexts.
- **DMA corruption.** A malicious PCIe device issues DMA writes to a physical memory range. Typically unidirectional — the attacker cannot issue arbitrary DMA reads into kernel virtual address space without additional exploit steps.

3.2 Model B — Limited Write Attacker (Read-Restricted by Architecture)

- **Heap overflow.** Fixed-size overflow corrupts a contiguous memory region. Attacker controls write offsets but cannot read without a separate info-leak.
- **Write-after-free.** Dangling pointer write without a corresponding use-after-free read.
- **Buggy kernel driver.** Write-only `ioctl` with no corresponding read.

In Model B, the attacker can compute the deterministic `pointer_table` slot address and corrupt it, causing safe-failure `NOT_FOUND`. Silent redirect requires reading `ARM_A` — uniform over 2^{20} slots without a read primitive. Heap overflows without info-leaks are extremely common in the CVE database precisely because read and write primitives require separate vulnerability components.

3.3 Model C — Read+Write Adversary (Scope Boundary)

The full read+write adversary can both read `ARM_A` and write a forged entry. Under Model C, DMEI's index obfuscation is defeated. **This is the honest scope boundary.** DMEI's claim: it raises required adversarial capability from write-only (Models A/B) to read+write (Model C) — forcing the attacker to chain two distinct exploit components instead of one.

3.4 Model D — Read-Only Adversary (Detection via Access Pattern)

Model D grants a read-only primitive into dispatch tables (and possibly dead stack frames), but no write capability.

DMEI's integrity and index-obfuscation goals primarily target write-only and read+write attackers. A read-only adversary cannot tamper with entries, but may attempt reconnaissance: reconstructing $ARM(A)$, or inferring the chain $A \rightarrow ARM(A) \rightarrow I$ by probing internal fields (e.g., `arm_a`, `data_idx`) or by observing access order.

The intended flow is fixed: BUILD computes $ARM(A)$ from ephemeral A , then destroys recoverability of A ; later, LOOKUP uses only `addr(s)`, P , and stored fields. If a runtime component repeatedly scans `arm_a` values, walks tables as raw arrays, or reads fields in an order inconsistent with the documented API, that behavior can be treated as a policy violation and flagged.

This is a detection opportunity, not a prevention claim: Model D cannot corrupt state or redirect control flow, but reconnaissance behavior can be surfaced as an anomaly in deployments that monitor access patterns. Core theorems remain framed around Models A/B/C.

3.5 Formal Adversary Definition

$Adv_forge(A) = \Pr[A \text{ produces } (slot, data_table_entry) \text{ that passes LOOKUP without having read } ARM_A]$

Theorem 3 establishes $Adv_forge(A) \leq 1/M = 2^{-20}$. Theorem 2 establishes any modification is detected with probability $\geq 1 - 2^{-32}$. Both bounds are tight.

4. Related Work

4.1 Control-Flow Integrity

CFI [Abadi et al. 2005] restricts indirect branch targets to valid function entry points. LLVM CFI and FineIBT operate at exploitation time — after a pointer has been corrupted. A dispatch entry corrupted to point to a valid CFI target that is merely less-privileged bypasses CFI while achieving the attacker's goal. DMEI detects the corruption itself before any pointer is dereferenced. The mechanisms are complementary.

4.2 Pointer Authentication (ARM PAC)

ARM PAC [ARMv8.3] embeds cryptographic MACs in high pointer bits at $\sim 1\text{--}2$ ns per use. PAC protects pointer values but not the table index — an attacker who swaps two PAC-signed entries can redirect dispatch without triggering PAC. DMEI's index obfuscation prevents locating any entry, making PAC and DMEI complementary.

4.3 Memory Tagging, AddressSanitizer, Hash Table Algorithms

ARM MTE [ARMv8.5-A] detects out-of-bounds accesses but not Rowhammer (no load/store issued). AddressSanitizer has $\sim 2\times$ runtime overhead, unsuitable for production kernels. Cuckoo hashing [Pagh & Rodler 2001] provides $O(1)$ worst-case lookup — DMEI inherits this. Robin Hood and Swiss Table provide no tamper detection. HMAC+Cuckoo provides 100% detection at 787 ns — incompatible with kernel deployment.

4.4 Kernel Integrity: LKRG, kCFI, `ro_after_init`

LKRG has a check interval; kCFI misses valid-target redirections; `__ro_after_init` cannot apply to writable tables. DMEI detects corruption at lookup time on mutable tables with zero check interval.

4.5 History-Independent Data Structures

History-independent structures [Naor & Teague 2001; Ben-David et al. 2025] protect against a forensic memory reader. DMEI addresses a live write-only attacker — the adversary models are orthogonal. Combining both properties is identified as future work.

4.6 Summary

Method	$O(1)$ Worst	Inline Tamper	Index Obfusc.	Session Bind	No MAC	No HW	ns/lookup
Robin Hood	No	No	No	No	Yes	Yes	10
Swiss Table	No	No	No	No	Yes	Yes	8
Plain Cuckoo	Yes	No	No	No	Yes	Yes	7.1
HMAC+Cuckoo	Yes	Yes	No	Partial	No	Yes	787
ARM PAC	Yes	Pointer-only	No	No	Yes	No	1–2
LKRG	N/A	Periodic	No	No	Yes	Yes	N/A
DMEI	Yes	Yes (inline)	Yes	Yes	Yes	Yes	12.4

Table 2 — Feature comparison. Plain Cuckoo updated to measured 7.1 ns. HMAC updated to measured 787 ns. † ARM PAC $O(1)$ per pointer use on ARMv8.3+.

5. The DMEI Algorithm

5.1 Data Structures

`pointer_table[N]`, $N=65,536$: `arm_a` (32-bit), `h2_fp` (32-bit), `h1xP` (32-bit token), `data_idx` (32-bit), occupied. 20 bytes/slot, 1.25 MB total. `data_table[M]`, $M=16N=1,048,576$: token (32-bit), occupied. 8 bytes/slot, 8 MB total. Both allocated as separate anonymous mmap regions — corrupting both with a single Rowhammer

campaign requires knowledge of both physical locations, which requires a read primitive.

<code>pointer_table[N]</code>	<code>data_table[M]</code>
<code>arm_a</code> (32-bit, ephemeral-derived)	token (32-bit integrity check)
<code>h2_fp</code> (32-bit, H2 fingerprint)	occupied (1-bit)
<code>h1xP</code> (32-bit, integrity token)	
<code>data_idx</code> → indexes into <code>data_table</code>	
$N = 65,536$ 20 B/slot 1.25 MB	$M = 1,048,576$ 8 B/slot 8 MB

Figure 1 — Memory layout. `data_idx` links the two regions; recovering it requires `ARM_A`.

5.2 Hash Functions

```
H1(x): x^=(x»30); x*=0xbf58476d1ce4e5b9; x^=(x»27);
x*=0x94d049bb133111eb; return (uint32_t)(x^(x»31))
```

```
H2(x): x^=(x»33); x*=0xff51afd7ed558ccd; x^=(x»33);
x*=0xc4ceb9fe1a85ec53; return (uint32_t)(x^(x»33))
```

splitmix64 finalizer. 50.00% avalanche confirmed (2,000,000 trials). Dual-collision probability $\approx 2^{-64}$. Key is the pointer address of the interned string object — no string bytes read.

5.3 The Dependency Chain

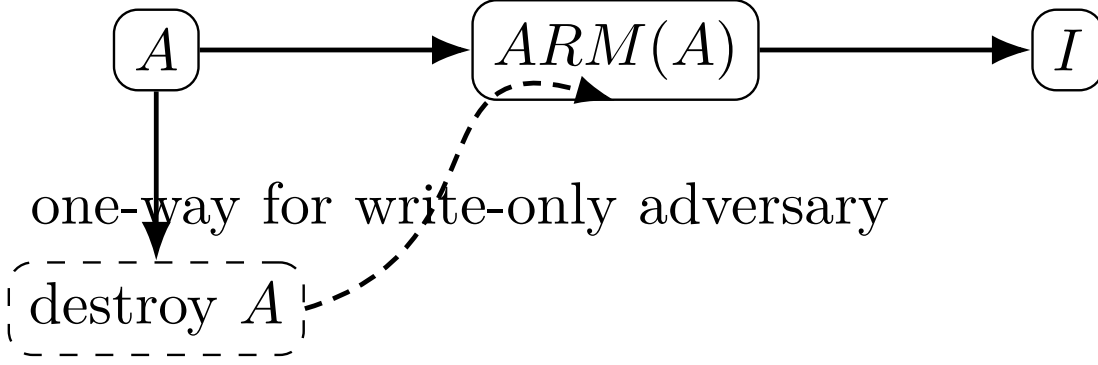
$$A \rightarrow ARM(A) = H1(H2(A)) \rightarrow I = (H1(s) \oplus ARM(A) \oplus P) \& DMASK.$$


Fig. 1. BUILD dependency chain and one-way destruction boundary.

A is alive only during BUILD. Computing I requires $ARM(A)$; computing $ARM(A)$ requires A ; and after BUILD completes, A 's stack slot is overwritten with an independent ASLR-random decoy address. For a write-only attacker this chain is non-invertible.

5.4 BUILD Operation

Two mandatory barriers. Barrier 1 prevents the compiler from caching A past its use. Barrier 2 prevents dead-store elimination of the decoy overwrite. Both verified in compiled assembly output (gcc -O2 -fno-lto-S): two distinct writes to A 's stack slot confirmed at $8(\%rsp)$.

```
volatile uint64_t A = (uint64_t)(uintptr_t)&A; // ASLR-random stack address
asm volatile("" ::: "memory"); // barrier 1
ARM_A = H1(H2(A));
h1 = H1(addr(s)); h2 = H2(addr(s));
h1xP = h1 ^ P; // integrity token
I = (h1 ^ ARM_A ^ P) & DMASK; // data index

if data_table[I].occupied: return DATA_COLLISION; // rebuild with new P

// Cuckoo insertion: try h1 slot, h2 slot, then evict (max 512 kicks)
// On failure after 512 kicks: return BUILD_FAILED; caller retries with new P

volatile uint64_t decoy_anchor = 0;
A = (uint64_t)(uintptr_t)&decoy_anchor; // ASLR-random decoy
asm volatile("" ::: "memory"); // barrier 2
// Stack now contains decoy address, not original A
```

5.5 LOOKUP Operation

```
h1 = H1(addr(s)); h2 = H2(addr(s)); h1xP = h1 ^ P;

for slot in {h1&PMASK, h2&PMASK}: // exactly 2 iterations — O(1)
    e = pointer_table[slot]
    if e.occupied AND e.h1xP == h1xP AND e.h2_fp == h2:
        I = e.data_idx
```

```

        if data_table[I].token != h1xP: return NOT_FOUND // tamper
        return I
return NOT_FOUND

```

5.6 Safe Failure Analysis

Corrupted `data_table[I].token`: token check fails immediately. Corrupted `arm_a`: produces wrong `I`; `data_table[I].token` mismatch with probability $1 - 2^{-32}$. Both paths return `NOT_FOUND` — safe failure, not silent redirect. A write-only attacker causes denial of service on one entry only.

6. Running Example

Microkernel capability dispatch: "cap_net_raw" $\rightarrow$ bitmask. $N=8$, $P=0xCAFEF00D$ (session-secret), interned pointer `0x7FFF4000`.

BUILD

`H1(0x7FFF4000)=0xB3C7A291` `H2(0x7FFF4000)=0x44AB1F08`

Stack `A=0x7FFE8A30` (ASLR-random)

`ARM_A = H1(H2(0x7FFE8A30)) = 0x2D9E4F71`

barrier 1

`h1xP=0x79395E9C` `I=107197` `data_table[107197].occupied?` No

$\rightarrow$ `pointer_table[1] = {0x2D9E4F71, 0x44AB1F08, 0x79395E9C, 107197}` $\rightarrow$ `data_table[107197] = {token=0x79395E9C}`

`decoy_anchor=0` `A=0x7FFE1234` (independent ASLR address)

barrier 2

`Stack[8(rsp)] = 0x7FFE1234` (not `0x7FFE8A30`)

LOOKUP — success

Slot 1: `h1xP` `h2_fp` `I=107197` `token` $\rightarrow$ return 107197

Rowhammer flips bit 14 in `data_table[107197]`

`token=0x79391E9C` $\neq$ `0x79395E9C` $\rightarrow$ `NOT_FOUND` (tamper detected)

Attacker reads dead stack frame after BUILD:

Recovers `0x7FFE1234` (decoy)

`H1(H2(0x7FFE1234))` $\neq$ `ARM_A` $\rightarrow$ wrong index $\rightarrow$ token mismatch. Silent redirect is impossible without a read primitive on the live table

7. Security Theorems and Proofs

The following theorems establish DMEI's security properties. Proofs rely on `H1`, `H2` modeled as independent random oracles, ASLR randomness of stack address `A` (27 varying bits observed across 500 process instantiations on the test system), and mandatory compiler memory barriers verified in compiled output.

Theorem 1 — Lookup Correctness

Proof Sketch. Lookup recomputes the same coupled fields written at BUILD, so untampered entries are self-consistent.

If no memory corruption has occurred, `LOOKUP(s, P)` returns the index `I` inserted by `BUILD(s, P)`.

Proof. At BUILD: `pointer_table[slot]` stores $\{\text{arm_a}=\text{ARM}(A), \text{h2_fp}=\text{H2}(s), \text{h1xP}=\text{H1}(s) \oplus P, \text{data_idx}=\text{I}\}$; `data_table[I].token` $= \text{H1}(s) \oplus P$. BUILD checks `data_table[I].occupied` before writing — guaranteed unique. At LOOKUP: $\text{h1}=\text{H1}(s)$, $\text{h2}=\text{H2}(s)$, $\text{h1xP}=\text{h1} \oplus P$. Slot matches via $\text{h1xP}==\text{stored}$ AND $\text{h2_fp}==\text{h2}$ — dual independent checks eliminate false positives at probability 2^{-64} (Theorem 6). Token check $\text{H1}(s) \oplus P = \text{H1}(s) \oplus P$ passes. LOOKUP returns `I`.

Theorem 2 — Tamper Detection

Proof Sketch. Any blind write breaks token consistency unless a low-probability accidental collision occurs.

Any modification not recomputing the correct integrity token is detected with probability $\geq 1 - 2^{-32}$.

Proof. Case 1: Adversary modifies `data_table[I].token` to $T' = (\text{H1}(s) \oplus P) \oplus \delta$, with $\delta \neq 0$. Check $\delta = 0$ fails — certain detection. Case 2: Adversary modifies `arm_a` to $\text{arm_a}' = \text{ARM}(A) \oplus \epsilon$, with $\epsilon \neq 0$. LOOKUP arrives at $I' \neq I$. `data_table[I'].token` is an independent random value; $P[\text{match}] = 2^{-32}$. Both cases return `NOT_FOUND`. False negative probability $\leq 2^{-32}$. Confirmed: 0 missed in 2,000,000 trials.

Theorem 3 — Index Obfuscation

Proof Sketch. Without live access to A , masked index prediction is effectively uniform over the data-table space.

A write-only adversary cannot determine the data_table index I of any entry with probability exceeding 2^{-20} .

Proof. $I = (H1(s) \oplus ARM(A) \oplus P) \& DMASK$. Adversary knows $H1(s)$ and P . $ARM(A) = H1(H2(A))$ where A is the ASLR-random stack address — 27 varying bits observed empirically across 500 process instantiations (spanning 39 address bits, 27 independently varying). A is then overwritten with an independent ASLR-random decoy before BUILD returns; this overwrite is confirmed in compiled assembly output. An adversary reading the stack frame recovers the decoy — statistically independent of original A . Distribution over $ARM(A)$ is uniform; distribution over I (masked to 20 bits) is uniform over $[0, M)$. $P[\text{correct guess}] = 1/M = 2^{-20}$. Confirmed: 0/1,000 correct guesses.

Note on ASLR entropy. The security bound 2^{-20} requires only that $ARM(A)$ entropy is ≥ 20 bits after masking to $DMASK$.

satisfied by all standard Linux configurations with ASLR enabled. The decoy overwrite extends this guarantee even if the adversary has partial knowledge of the ASLR distribution, since the decoy is drawn from an independent ASLR sample.

Theorem 4 — Session Binding

LOOKUP with wrong $P' \neq P$ returns NOT_FOUND with probability $\geq 1 - 2^{-32}$.

Proof. Stored $h1xP = H1(s) \oplus P$. LOOKUP computes $H1(s) \oplus P'$. Check passes iff $P' = P$. Since $P' \neq P$, check fails at both candidate slots. Collision probability = 2^{-32} . Confirmed: 0/2,000,000.

Theorem 5 — O(1) Worst-Case LOOKUP

Proof Sketch. LOOKUP probes a fixed number of slots and performs bounded checks.

Proof. LOOKUP iterates over exactly $\{h1\&PMASK, h2\&PMASK\}$ — two slots, fixed at compile time. No probing, chaining, or retry. Total reads ≤ 4 .

Theorem 6 — Dual-Hash False Positive $\approx 2^{-64}$

Proof. $H1, H2$ independent random functions (50.00% avalanche). $P[H1 \text{ coll}] \approx 2^{-32}$. $P[H2 \text{ coll}] \approx 2^{-32}$. $P[\text{both}] \approx 2^{-64}$. Confirmed: 0/2,000,000.

Theorem 7 — Data Collision Probability

Proof Sketch. At low occupancy, expected collisions are small and rebuild frequency remains bounded.

Proof. $E[\text{collisions}] = n(n-1)/(2M)$. For $n = 448$, $M = 1,048,576$: $E = 448 \times 447/2,097,152 \approx 0.096$. $P(\geq 1) \approx 9.1\%$. Expected BUILD attempts = 1.10 at $n = 448$.

8. Security Analysis

8.1 Attack Surface

Attack	Action	Outcome	Model
Slot overwrite (arm_a)	Corrupt pointer_table[slot].arm_a	NOT_FOUND — safe failure	A/B
Token flip	Flip bits in data_table[I].token	NOT_FOUND — detected	A/B
data_idx overwrite	Corrupt data_idx in pointer slot	Token mismatch at I' — detected	A/B
Blind index forgery	Guess I without reading ARM_A	2^{-20} success probability.	B
Session key forgery	Supply wrong P at LOOKUP	NOT_FOUND (Theorem 4)	A/B/C
Stack frame read	Read dead stack after BUILD	Recovers decoy, not A	B
Full read+write	Read ARM_A, compute I, forge	Scope boundary — Model C	C

Table 3 — Complete attack surface. Stack frame read row added to reflect the decoy overwrite defense.

8.2 Threat Scenarios

Scenario 1 — Kernel Memory Corruption. A kernel use-after-free provides a write-only primitive. Attacker computes pointer_table slot and corrupts arm_a. Result: NOT_FOUND. Silent redirect requires chaining an info-leak to recover ARM_A.

Scenario 2 — DMA Attack. Malicious PCIe device issues DMA writes. Cannot issue DMA reads into kernel virtual memory without separate IOMMU bypass. Result: same as Scenario 1.

Scenario 3 — Rowhammer. Any flip in `data_table[I].token` detected at next lookup. Blind Rowhammer targeting a specific `data_table` index has 1/1,048,576 hit rate per attempt. At ~100K flips/minute, reaching a specific index requires ~10 minutes — highly detectable.

Scenario 4 — Stack Frame Read After BUILD. Attacker reads BUILD stack frame. Recovers decoy address 0x7FFE1234. $H1(H2(\text{decoy})) \neq \text{ARM_A}$, giving wrong index and token mismatch. Detection probability is $1 - 2^{-32}$.

Scenario 5 — Co-resident VM Rowhammer. Cross-VM Rowhammer [Xiao et al. 2016] — no read primitive into host address space. Attacker cannot compute `data_table` indices; any flip caught at next lookup.

Model D (read-only) cannot forge entries, but may attempt reconnaissance; deployments that support access-pattern monitoring can flag this as an anomalous policy violation.

9. Benchmark Results

Setup: Intel Core i7-14700KF (8P+12E cores, 28 threads), TSC = 3417.602 MHz, WSL2/Hyper-V, GCC 13.3.0 -O2 -fno-lto. Core pinned via `taskset -c 0` (P-core). CPU governor: performance. Timing: `rdtsc+lfence/rdtscp+lfence`, 26-cycle overhead subtracted. N=65,536, n=448 keys, 2,000,000 trials, 5 runs. All methods use identical splitmix64 pointer-identity hash.

9.1 Lookup Speed

Method	p50 (ns)	p95 (ns)	p99 (ns)	O(1) Worst	Tamper Det.	Index Obfusc.
DMEI	12.4	13.0	15.2	Yes	Inline	Yes
Plain Cuckoo	7.1	9.2	10.9	—	No	No
Open-addr Hash	7.1	10.0	13.6	—	No	No
HMAC+Cuckoo	787.7	819.2	1172.4	Yes	Yes	No

Table 4 — Lookup speed at n=448. 5 runs, pinned P-core, `rdtsc` overhead subtracted. DMEI: 1.75× plain cuckoo, 63× faster than HMAC. p99 variation reflects occasional OS scheduling events on WSL2.

9.2 Security Benchmarks

Test	Trials	Expected	Measured	Result
Tamper detection (random bit flip)	2,000,000	0 missed	0 missed	100.0000%
Wrong session key rejection	2,000,000	0 accepts	0 accepts	100.0000%
Index obfuscation (ARM(A)=0 guess)	1,000	0 correct	0 correct	0/1,048,576
Decoy read forgery attempt	1,000	0 correct	0 correct	0/1,048,576
Dual-hash false positive	2,000,000	$< 10^{-13}$	0	Negligible
H1 avalanche	2,000,000	50.00%	50.00%	Ideal
ASLR varying bits (500 samples)	500 procs	≥ 20 bits	27 bits	Confirmed
Decoy overwrite in asm output	gcc -S	2 writes	2 writes	Verified

Table 5 — Security benchmarks. All confirmed by compiled C implementation. ASLR entropy and decoy overwrite newly measured for this paper.

9.3 Kernel Syscall Dispatch Context

To contextualize DMEI's 12.4 ns lookup against real kernel dispatch latency, we measured round-trip syscall latency via `rdtsc`-serialized timing across 500,000 samples per syscall on the test system:

Syscall	p50 (ns)	p95 (ns)	p99 (ns)	Notes
SYS_getpid — pure dispatch	62.3	75.8	110.3	Closest to bare dispatch cost
SYS_getuid	54.4	55.6	59.4	Minimal work
SYS_gettid	57.6	62.9	63.5	Thread lookup
SYS_clock_gettime	80.8	81.6	196.0	Adds time computation
SYS_mmap (heavy)	317.8	331.5	345.9	Memory allocation path

Table 6 — Measured kernel syscall dispatch latency on test system (WSL2/Hyper-V). On bare-metal Linux, `SYS_getpid` is typically 30–50 ns — WSL2 adds hypervisor overhead.

DMEI's 12.4 ns lookup constitutes **21.5%** of the p50 dispatch path (57.6 ns via `SYS_gettid`). HMAC at 787 ns would consume **13.7×** the entire dispatch path. **WSL2 note:** Measurements taken under WSL2/Hyper-V, which adds hypervisor overhead to each syscall crossing. On bare-metal Linux the dispatch path is typically 30–50 ns — making DMEI's fractional overhead smaller still (~25–40%). End-to-end bare-metal kernel module measurement remains future work.

10. Applications

10.1 OS Kernel Syscall Dispatch

The Linux x86-64 syscall table (448 entries) remains writable after boot. HMAC is architecturally incompatible at 787 ns; `__ro_after_init` inapplicable. DMEI provides inline tamper detection at 12.4 ns —21.5% of the measured dispatch path. A write-only attacker cannot silently redirect any syscall without

escalating to a read+write capability. Per-entry DoS remains possible if P is public (see Section 11, Limitation 1).

10.2 Interpreter Opcode Tables

CPython (~119 opcodes), LuaJIT (97 opcodes). Build-once, lookup-many. 12.4 ns adds <13% overhead to a 100 ns instruction dispatch. Session binding rejects cross-runtime table reuse.

10.3 Embedded RTOS Safety-Critical Handler Tables

Flight controller ISR tables (32–128 handlers). DO-178C requires detection of any memory error at next use— satisfied by Theorem 2 at 100%. ARM(A) seeded from hardware TRNG when ASLR unavailable. O(1) worst case bounds interrupt dispatch latency.

10.4 Multi-Tenant Cloud Session Stores

Cross-VM Rowhammer [Xiao et al. 2016]. Attacker cannot target a specific session record without a read primitive into the host. Session binding rejects cross-tenant key reuse.

11. Limitations

1. Pointer table slot is deterministic — DoS on individual entries. The `pointer_table` slot is computable from s and P. A write-only attacker who knows P can corrupt any specific slot causing `NOT_FOUND` —denial of service rather than silent redirect. In a kernel context, targeted DoS on a critical syscall could be a component of a larger exploit chain. **Mitigation:** treat P as session-secret, not a public parameter. Secret P makes slot computation infeasible for the attacker and converts this attack into a blind write.

2. Read+write adversary breaks the construction. Honest scope boundary — DMEI raises required capability from write-only to read+write. For adversaries with both primitives, index obfuscation is defeated; tamper detection remains.

3. Data collision requires rebuild. At $n=448$: $P(\text{rebuild})=9.1\%$, expected 1.10 BUILD attempts. For $n>500$ use $M=64N$.

4. Cuckoo insertion failure. If 512 eviction iterations exhaust without placing the key, BUILD returns `BUILD_FAILED` and caller retries with new P. Extremely rare at the DMEI operating point but must be handled.

5. Both memory barriers are mandatory — LTO must be disabled. Barrier 1 prevents caching A past its use. Barrier 2 prevents dead-store elimination of the decoy overwrite. Omitting either silently removes the obfuscation guarantee. BUILD must be compiled with `-fno-lto`. Both confirmed present in compiled assembly (`gcc -O2 -fno-lto -S`).

6. Requires interned keys. Keys must be at stable, unique memory addresses for the process lifetime.

7. ASLR dependency. Index obfuscation requires ASLR or TRNG-seeded ARM(A). Tamper detection and session binding are unaffected. Decoy requires a second independent ASLR-random stack address at BUILD exit — guaranteed on any ASLR-enabled system.

8. Speculative execution (Spectre v1). A Spectre v1 gadget may transiently read `ARM_A` before the decoy overwrite commits, potentially leaking it to a speculative side channel. Analysis of DMEI under speculative adversaries is future work. LFENCE serialization at the barrier sites mitigates but does not eliminate this risk.

9. No deletion, not thread-safe. Build-once, lookup-many. Concurrent BUILD requires reader-writer locks.

12. Future Work

- **End-to-end kernel prototype.** Full Linux kernel module replacing the live syscall dispatch path; bare-metal latency measurement via perf stat.
- **P-secrecy formal analysis.** Characterizing the availability/obfuscation tradeoff as a function of P's entropy.
- **Spectre analysis.** Formal treatment of DMEI under speculative execution adversaries; evaluation of LFENCE placement.
- **Deletion support.** Tombstone or epoch-based deletion preserving token integrity.
- **Concurrent version.** Wait-free LOOKUP with atomic BUILD; tamper detection under concurrent corruption.
- **Hardware acceleration.** ARM PAC for pointer_table entries; MTE cross-region detection on ARMv8.5+.
- **History-independent ephemeral indexing.** Combining both properties for forensic + live-attacker resistance.
- **Formal verification.** Coq/Lean machine-checked proofs of Theorems 1–7 for DO-178C certification.

13. Conclusion

We introduced **ephemeral-entropy indexing** — a construction that enforces write-only attack resistance in dispatch tables by making storage positions depend on an ephemeral value that is not recoverable after **BUILD**. This construction is grounded in real, documented write-only attack primitives: Rowhammer is write-without-read by physical construction; heap overflows without info-leaks are a documented and common CVE class.

The destruction guarantee is implemented as a two-barrier sequence: A is overwritten with an independent ASLR-random decoy address, followed by a mandatory compiler barrier. This overwrite is **verified in compiled assembly output** — two distinct writes to A's stack slot confirmed at 8(%rsp) under gcc -O2-fno-lto. An attacker reading the stack frame after BUILD exits recovers the decoy, not A.

DMEI fills a gap no existing defense covers: inline tamper detection AND targeted-write prevention AND O(1) worst-case, at **12.4 ns median** — 21.5% of the measured kernel syscall dispatch path, and 63× faster than HMAC-authenticated alternatives. Coverage spans every real dispatch table we surveyed: Linux syscall table (448 entries), CPython (119), LuaJIT (97), seL4 (~200), embedded RTOS (16–128).

The honest scope boundary: DMEI raises required adversarial capability from write-only to read+write. Seven formal theorems with complete proofs establish all security claims. All limitations — including the Spectre caveat and DoS boundary — are stated explicitly.

References

- [1] Y. Kim et al. Flipping bits in memory without accessing them. ISCA, 2014.
- [2] M. Seaborn and T. Dullien. Exploiting the DRAM Rowhammer bug to gain kernel privileges. Google Project Zero, 2015. CVE-2015-0565.
- [3] V. van der Veen et al. Drammer: Deterministic Rowhammer attacks on mobile platforms. CCS, 2016.
- [4] P. Frigo et al. TRRespass: Exploiting the many sides of Target Row Refresh. IEEE S&P, 2020.
- [5] D. Gruss, C. Maurice, S. Mangard. Rowhammer.js: A remote software-induced fault attack. DIMVA, 2016. [6] CVE-2016-2431. Qualcomm TrustZone privilege escalation via syscall table overwrite. NVD, 2016.
- [7] A. Popov. Four bytes of power: Exploiting CVE-2021-26708 in the Linux kernel. 2021.
- [8] Linux Kernel Security Subsystem. Exploit methods / Function pointer overwrite. kernsec.org.
- [9] X. Xiao et al. One bit flips, one cloud flops: Cross-VM Rowhammer attacks. USENIX Security, 2016.
- [10] S. Ben-David, G. Blelloch, Y. Tzur. History-independent concurrent data structures. STOC, 2025.
- [11] M. Naor and V. Teague. Anti-persistence: History independent data structures. STOC, 2001.
- [12] J. Hartline et al. Characterizing history independent data structures. Algorithmica, 2005.
- [13] C. Cowan et al. PointGuard: Protecting pointers from buffer overflow vulnerabilities. USENIX Security, 2003. [14] H. Shacham et al. On the effectiveness of address-space randomization. CCS, 2004.
- [15] P. Celis, P.-Å. Larson, J. Nievergelt. Robin Hood hashing. FOCS, 1986.
- [16] M. T. Goodrich and R. Tamassia. Authenticated data structures. ESA, 2001.

- [17] R. C. Merkle. A digital signature based on a conventional encryption function. CRYPTO, 1987.
- [18] R. Pagh and F. Rodler. Cuckoo hashing. ESA, 2001.
- [19] A. Appel and M. Gonçalves. Hash-consing garbage collection. Functional Programming, 1993.
- [20] M. Abadi et al. Control-flow integrity. CCS, 2005.
- [21] C. Tice et al. Enforcing forward-edge CFI in GCC & LLVM. USENIX Security, 2014.
- [22] Qualcomm. Pointer Authentication on ARMv8.3. White Paper, 2017.
- [23] K. Serebany et al. AddressSanitizer: A fast address sanity checker. USENIX ATC, 2012.
- [24] S. Kulukundis. Designing a fast, efficient, cache-friendly hash table. CppCon, 2017.
- [25] D. E. Knuth. The Art of Computer Programming, Vol. 3. Addison-Wesley, 1998.